

REACT

1. What is React?

React is a JavaScript library for building user interfaces, especially component-based and interactive web applications. It allows us to break the UI into reusable components and efficiently update the UI when state or props change.

What is a component?

A component is a reusable piece of UI.

```
function UserCard({ name }) {  
  return <h2>{name}</h2>;  
}
```

Components can have:

- Props
- State
- Event handlers
- Hooks
- Child components

2. What are props?

Props are values passed from a parent component to a child.

```
function Parent() {  
  return <Child name="Avhi" />;  
}
```

```
function Child({ name }) {  
  return <h1>Hello {name}</h1>;  
}
```

Props are **read-only** from the child's perspective.

What is state?

State is data managed by a component that can change over time.

```
const [count, setCount] = useState(0);
```

When state changes, React schedules a re-render.

Component lifecycle

For functional components, think of the lifecycle as:

```
Mount  
↓  
Render  
↓  
Effects  
↓  
Update  
↓  
Render  
↓  
Effects  
↓  
Unmount
```

useEffect is commonly used for side effects and cleanup.

```
useEffect(() => {  
  const subscription = subscribe();  
  
  return () => {  
    subscription.unsubscribe();  
  };  
}, []);
```

The cleanup runs when the component unmounts and before the effect runs again when dependencies change.

3. React Hooks

Hooks allow functional components to use React features such as state and effects.

useState

Ans:

useState is a React Hook that allows a functional component to store and manage local state. When the state setter is called with a new value, React schedules a re-render so the UI can reflect the updated state.

Syntax

```
const [state, setState] = useState(initialValue);
```

Example:

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <button onClick={() => setCount(count + 1)}>  
      {count}  
    </button>  
  );  
}
```

Here:

count → current state

setCount → function used to update state

0 → initial value

Updating based on previous state

If the new state depends on the previous state, use the functional updater:

```
setCount(prevCount => prevCount + 1);
```

This is especially important when multiple updates may be queued.

```
setCount(prev => prev + 1);
```

```
setCount(prev => prev + 1);
```

The state increases by 2.

useState is used to manage local state in functional components. It returns the current state value and a setter function. Calling the setter schedules a re-render. When the next state depends on the previous state, I use the functional updater form to ensure I work with the latest state.

2. useEffect

useEffect is used to synchronize a component with external systems.

Examples include:

- API requests
- Timers
- Event listeners
- WebSocket subscriptions
- Browser APIs
- Third-party libraries

```
useEffect(() => {  
  // side effect  
}, []);
```

Dependency array

No dependency array

```
useEffect(() => {  
  console.log("Effect");  
});
```

Runs after every completed render.

Empty dependency array

```
useEffect(() => {  
  console.log("Effect");  
}, []);
```

Runs after the initial mount in normal production behavior.

In development with React Strict Mode, React may intentionally run setup/cleanup an extra time to help identify unsafe effects.

With dependencies

```
useEffect(() => {  
  fetchUser(userId);  
}, [userId]);
```

The effect runs after the initial render and again when userId changes.

Cleanup

An effect can return a cleanup function:

```
useEffect(() => {  
  const timer = setInterval(() => {  
    console.log("Running");  
  }, 1000);
```

```
  return () => {  
    clearInterval(timer);  
  };  
}, []);
```

Cleanup happens when the component is removed and before the effect is re-run because dependencies changed.

API example

```
useEffect(() => {  
  const fetchUsers = async () => {  
    const response = await fetch("/api/users");  
    const data = await response.json();
```

```
    setUsers(data);  
  };
```

```
    fetchUsers();  
  }, []);
```

In production applications, you should also consider:

- loading state
- error handling
- cancellation
- caching
- race conditions

Interview answer

useEffect is used to synchronize a component with external systems such as APIs, subscriptions, timers, browser APIs, or third-party libraries. The dependency array controls when the effect is re-synchronized, and the cleanup function is used to remove subscriptions, timers, listeners, or other resources.

3. useContext

useContext allows a component to read a value from React Context without manually passing that value through every intermediate component.

Without Context:

App

↓

Parent

↓

Child

↓

GrandChild

You may have to pass props through every level.

This is called prop drilling.

Context can provide the value directly to consumers.

Example

Create context:

```
const ThemeContext = createContext(null);
```

Provide value:

```
<ThemeContext.Provider value="dark">
```

```
  <App />
```

```
</ThemeContext.Provider>
```

Consume it:

```
const theme = useContext(ThemeContext);
```

Common use cases

- Theme
- Authentication/user information
- Language
- Application configuration

Important interview point

Context is not automatically a replacement for Redux.

Context is a mechanism for passing values through the tree.
Redux and other state-management libraries provide additional patterns and tooling for managing complex application state.

4. useReducer

useReducer is useful for managing complex state logic, especially when state updates involve multiple actions or related pieces of state.

It works similarly to the reducer concept used in Redux.

Syntax

```
const [state, dispatch] = useReducer(reducer, initialState);
```

Example:

```
function reducer(state, action) {  
  switch (action.type) {  
    case "increment":  
      return {  
        ...state,  
        count: state.count + 1  
      };  
  
    case "decrement":  
      return {  
        ...state,  
        count: state.count - 1  
      };  
  
    default:  
      return state;  
  }  
}
```

Component:

```
function Counter() {  
  const [state, dispatch] = useReducer(reducer, {  
    count: 0  
  });  
  
  return (  
    <>  
      <p>{state.count}</p>  
  
      <button onClick={() => dispatch({ type: "increment" })}>  
        +  
      </button>  
  
      <button onClick={() => dispatch({ type: "decrement" })}>  
        -  
      </button>  
    </>  
  );  
}
```

Flow:

User action

↓
dispatch()
↓
Action
↓
Reducer
↓
New State
↓
Re-render

When should you use it?

Use `useReducer` when:

- State has multiple related fields
- There are many state transitions
- State update logic is becoming complicated
- You want explicit actions

For simple state:

`const [count, setCount] = useState(0);`
is usually easier.

5. `useRef`

`useRef` creates a mutable object whose `.current` property persists across renders without causing a re-render when changed.

`const ref = useRef(initialValue);`

DOM access

One common use case is accessing a DOM element:

```
function Input() {  
  const inputRef = useRef(null);  
  
  const focusInput = () => {  
    inputRef.current.focus();  
  };  
  
  return (  
    <>  
      <input ref={inputRef} />  
      <button onClick={focusInput}>  
        Focus  
      </button>  
    </>  
  );  
}
```

Storing values between renders

You can also use it for mutable values:

`const renderCount = useRef(0);`

`renderCount.current += 1;`

Changing:

renderCount.current
doesn't itself trigger a re-render.

useState vs useRef

useState	useRef
Stores state	Stores mutable reference
Updating causes re-render	Updating .current doesn't cause re-render
Used for UI state	DOM references / persistent mutable values

Interview answer

I use useRef when I need a value to persist across renders without triggering a re-render, or when I need to access a DOM element directly.

6. useMemo

Ans:

useMemo memoizes the result of a calculation.

```
const value = useMemo(() => {  
  return expensiveCalculation(data);  
}, [data]);
```

React can reuse the previously calculated value until a dependency changes.

Example

```
const filteredUsers = useMemo(() => {  
  return users.filter(user => user.active);  
}, [users]);
```

If another state changes but users doesn't, React can reuse the memoized result.

Important

Don't use useMemo everywhere.

This:

```
const total = useMemo(() => price * quantity, [price, quantity]);  
may provide no meaningful benefit if the calculation is trivial.
```

Interview answer

useMemo is used to memoize the result of an expensive calculation so React can reuse the previous result when its dependencies haven't changed. I use it when profiling or the calculation/reference behavior justifies it, rather than using it for every value.

7. useCallback

useCallback memoizes a function reference.

```
const handleClick = useCallback(() => {  
  console.log("Clicked");  
}, []);
```

This is particularly useful when passing callbacks to memoized child components.

Why?

Every render normally creates a new function:

```
function Parent() {  
  const handleClick = () => {  
    console.log("Hello");  
  };  
}
```

```
    return <Child onClick={handleClick} />;  
  }
```

Conceptually:

Render 1 → Function A

Render 2 → Function B

Render 3 → Function C

Even though the function looks identical:

Function A !== Function B

With useCallback:

```
const handleClick = useCallback(() => {  
  console.log("Hello");  
}, []);
```

React can preserve the function identity while dependencies remain unchanged.

useMemo vs useCallback

This is a very common interview question.

useMemo

↓

memoizes a value

useCallback

↓

memoizes a function

Example:

```
const result = useMemo(() => calculate(), [data]);
```

```
const handleClick = useCallback(() => {  
  doSomething();  
}, []);
```

8. useEffect

useLayoutEffect is similar to useEffect, but its effect runs synchronously after DOM mutations and before the browser paints the updated screen.

This makes it useful when you need to:

- Measure DOM dimensions
- Read layout information
- Make a DOM adjustment before the user sees the result

Example:

```
useLayoutEffect(() => {  
  const width = elementRef.current.getBoundingClientRect().width;
```

```
  console.log(width);  
}, []);
```

Difference

useEffect

DOM update

↓

Browser may paint

↓

Effect

useLayoutEffect

DOM update



Layout effect



Browser paint

Because useLayoutEffect can block painting, don't use it for ordinary effects when useEffect is sufficient.

4. Re-rendering & Reconciliation

1. What causes a React component to re-render?

Main causes:

1. Its state changes
2. Its props change
3. Its parent re-renders
4. A consumed context value changes

Example:

```
const [count, setCount] = useState(0);
```

```
setCount(count + 1);
```

The state update causes React to render the component again.

Does re-render mean the DOM is recreated?

No.

React calculates what changed and updates only the necessary parts of the actual DOM.

What is reconciliation?

Reconciliation is the process React uses to compare the previous UI representation with the new one and determine what needs to be updated.

Keys are particularly important when rendering lists.

```
users.map(user => (  
  <UserCard key={user.id} user={user} />  
))
```

A stable key helps React correctly identify elements between renders.

2. Virtual DOM & React Fiber

What is Virtual DOM?

The Virtual DOM is a lightweight in-memory representation of the UI that React uses to determine what changes need to be made to the actual DOM.

For example:

State changes



React renders



New UI representation



Compare with previous



Determine changes



Update DOM

3. What is React Fiber?

Fiber is React's internal reconciliation architecture. It represents work as units that React can schedule and prioritize, enabling more sophisticated rendering behavior.

It helps React handle rendering work more efficiently and supports capabilities such as concurrent rendering.

Interview tip

Don't say:

"Virtual DOM makes React automatically faster than every other framework."

Instead:

"React's reconciliation and scheduling mechanisms help minimize unnecessary DOM work and manage rendering efficiently."

5. Props, State & Context API

Props vs State

Props

Passed by parent

Read-only

Used to configure component

Changes when parent provides new value

State

Managed by component

Can be updated

Represents changing data

Changes through state updates

Example:

```
function User({ name }) {  
  const [age, setAge] = useState(25);
```

```
  return (  
    <>  
      <h2>{name}</h2>  
      <p>{age}</p>  
    </>  
  );  
}
```

name → prop

age → state

Context API

Context allows data to be shared with components without manually passing props through every level.

```
const ThemeContext = createContext();
```

```
<ThemeContext.Provider value="dark">
```

```
  <App />
```

```
</ThemeContext.Provider>
```

Then:

```
const theme = useContext(ThemeContext);
```

When would you use Context?

Good examples:

- Theme
- Authentication/user information
- Language
- Configuration

Avoid putting every piece of application state into Context because context updates can cause consumers to re-render.

6. Custom Hooks

What is a custom hook?

A custom hook is a JavaScript function starting with use that allows us to extract and reuse stateful React logic.

Example:

```
function useFetch(url) {  
  const [data, setData] = useState(null);  
  const [loading, setLoading] = useState(true);
```

```
  useEffect(() => {  
    fetch(url)  
      .then(res => res.json())  
      .then(result => {  
        setData(result);  
        setLoading(false);  
      });  
  }, [url]);
```

```
  return { data, loading };  
}
```

Use it:

```
function Users() {  
  const { data, loading } = useFetch("/api/users");  
  
  // ...  
}
```

Important interview point

A custom hook **shares logic, not state**.

If two components call:

`useFetch(...)`

each invocation has its own state unless the hook uses some shared external state mechanism.

7. Performance Optimization

This is one of the most important topics for your interview.

How do you optimize React?

I first identify the actual bottleneck using React DevTools Profiler, Chrome DevTools, Lighthouse, and the Network tab. Then I optimize based on the bottleneck.

Common techniques:

React.memo

Prevents a component from rendering when its props haven't changed.

```
const UserCard = React.memo(UserCardComponent);
```

useMemo

Memoizes expensive calculations.

useCallback

Maintains function identity when useful, especially when passing callbacks to memoized children.

Code splitting

```
const Dashboard = lazy(() => import("./Dashboard"));
```

Lazy loading

Load components or resources only when needed.

Image optimization

- Compress images
- Use modern formats
- Lazy-load where appropriate
- Use proper dimensions
- Next.js Image where applicable

Large lists

Use:

- Pagination
- Virtualization
- Infinite scrolling

API optimization

- Caching
- Debouncing
- Pagination
- Avoid duplicate requests

Bundle optimization

- Remove unused dependencies
- Code splitting
- Dynamic imports
- Analyze bundle size

Strong interview answer

I don't use memoization everywhere. I first profile the application, identify unnecessary renders or expensive operations, and then apply techniques such as React.memo, useMemo, useCallback, code splitting, lazy loading, image optimization, caching, and list virtualization where they actually provide a benefit.

8. State Management

State management means managing application data that changes over time.

Local state

Use:

useState()

for state needed by one component or a small component tree.

Context

Useful for relatively global values:

Theme

User

Language

Redux

Useful when an application has complex global state and predictable state transitions.

Example architecture

Component

↓

Action

↓

Reducer

↓

Store



Component

Redux vs Context

Context is primarily a mechanism for passing values through the component tree. Redux is a state management solution with a structured architecture for managing complex application state.

Don't say:

"Redux is always better."

Choose based on the application's requirements.

9. API Integration

How do you integrate APIs in React?

Example:

```
useEffect(() => {  
  const fetchUsers = async () => {  
    try {  
      setLoading(true);  
  
      const response = await fetch("/api/users");  
      const data = await response.json();  
  
      setUsers(data);  
    } catch (error) {  
      setError(error.message);  
    } finally {  
      setLoading(false);  
    }  
  };  
  
  fetchUsers();  
}, []);
```

I normally handle:

Loading

Success

Error

Empty state

Authentication

Retry

Caching

REST methods

GET → Read

POST → Create

PUT → Replace

PATCH → Partial update

DELETE → Delete

Example

GET /users

GET /users/10

POST /users

PATCH /users/10

DELETE /users/10

API error handling

Don't only handle 500.

You should understand common status codes:

200 → Success

201 → Created

400 → Bad Request

401 → Unauthenticated

403 → Forbidden

404 → Not Found

409 → Conflict

500 → Server Error

10. React Performance & Best Practices

These overlap with optimization, but interviewers may ask for **best practices** separately.

Component design

Keep components:

- Small
- Reusable
- Focused
- Easy to understand

Avoid huge components containing everything.

Use meaningful names

Instead of:

```
<Cmp />
```

use:

```
<UserProfile />
```

Don't duplicate logic

Extract repeated logic into:

- Custom hooks
- Utility functions
- Reusable components

Don't overuse global state

Keep state as close as possible to where it is needed.

Don't overuse useMemo / useCallback

They aren't automatically performance improvements.

Use them when reference stability or expensive computation makes them worthwhile.

Use stable keys

Good:

```
users.map(user => (  
  <User key={user.id} user={user} />  
))
```

Avoid using array index as key when list ordering can change.

Handle loading/error states

Don't assume APIs always succeed.

Keep secrets out of frontend code

Never expose sensitive credentials or private API keys in client-side JavaScript.

11. How would you design a React application?

For a medium-sized application, I might structure it like:

```
src/  
|  
├── components/  
|   ├── common/  
|   └── features/  
|  
├── pages/ or app/  
|  
├── hooks/  
|  
├── services/  
|  
├── utils/  
|  
├── types/  
|  
├── store/  
|  
└── styles/
```

And conceptually:

```
UI  
↓  
Components  
↓  
Hooks / State  
↓  
Services  
↓  
REST API  
↓  
Backend  
↓  
Database
```

12. Explain Redux

Ans: React-Redux is a tool that helps you manage the state (data) of your React app in one central place. This makes your app easier to maintain and debug as it grows.

It is based on three fundamental principles:

Single Source of Truth: All app data lives in the store.

State is Read-Only: You can't change data directly; you send an action to request a change.

Changes with Pure Functions: Reducers take the current state and an action, and return the updated state.

Core Concepts of React-Redux

To effectively use React-Redux, you need to understand some key concepts:

1. Store

The store is a central place that holds all the app's data. It's the only place where the data can be changed.

2. Actions

An action is a simple JavaScript object that tells Redux what change to make. It must have a type and can optionally include a payload with extra information.

```
const incrementAction = {  
  type: 'INCREMENT',  
  payload: 1  
};
```

3. Reducers

A reducer is a pure function that decides how the state changes when an action is received. It takes the current state and an action, and returns a new state.

```
const counterReducer = (state = 0, action) => {  
  switch (action.type) {  
    case 'INCREMENT':  
      return state + action.payload;  
    case 'DECREMENT':  
      return state - action.payload;  
    default:  
      return state;  
  }  
};
```

4. Dispatch

Dispatch is used to send an action to the Redux store. This triggers the reducer to update the state.

```
store.dispatch(incrementAction);
```

5. Selectors

A selector is a function that is used to get specific data from the Redux store.

```
const selectCount = (state) => state.count;
```

6. Provider

The Provider component makes the Redux store available to all React components in the app.

```
import { Provider } from 'react-redux';
```

```
<Provider store={store}>  
  <App />  
</Provider>;
```

7. connect()

connect() is used to connect a React component to the Redux store so it can access state and dispatch actions.

```
import { connect } from 'react-redux';
```

```
const Counter = ({ count, increment }) => (  
  <div>  
    <h1>{count}</h1>  
    <button onClick={increment}>Increment</button>  
  </div>
```

```

);

const mapStateToProps = (state) => ({
  count: state.count
});

const mapDispatchToProps = (dispatch) => ({
  increment: () => dispatch({ type: 'INCREMENT', payload: 1 })
});

export default connect(mapStateToProps, mapDispatchToProps)(Counter);

```

13. Explain Context Api

Ans: Context API is a React feature used to share data between multiple components without passing props through every intermediate component. It helps solve the problem of prop drilling.

We create a Context using `createContext()`, provide the data using a `Provider`, and access that data in child components using `useContext()`.

For example, I can use Context for **logged-in user information, authentication, theme, language, or global settings**.

Context itself doesn't manage state. We can combine it with `useState` or `useReducer` when we need to manage and share state.

Simple Example

```

const UserContext = createContext();

function App() {
  const user = { name: "Avhi" };

  return (
    <UserContext.Provider value={user}>
      <Dashboard />
    </UserContext.Provider>
  );
}

```

```

function Dashboard() {
  const user = useContext(UserContext);

  return <h2>{user.name}</h2>;
}

```

How it works

```

createContext()
  ↓
Provider
  ↓
Child Components
  ↓
useContext()
  ↓
Get Data

```

One important interview point

If the **Context value changes**, components consuming that Context may re-render. So I wouldn't put every piece of application state into one Context. For complex global state, a dedicated state-management solution such as Redux may be more appropriate.

"Context API allows us to share data across the component tree without prop drilling. We use createContext to create it, a Provider to supply the value, and useContext to consume it. I commonly use it for authentication, themes, and other shared application-level data."

14. What are Keys in React?

Ans:

Keys are unique identifiers that React uses to identify elements in a list. They help React understand which items were added, removed, or changed when the list is updated.

Keys are mainly used when rendering lists using .map().

Example

```
const users = [
  { id: 1, name: "Avhi" },
  { id: 2, name: "Rahul" },
  { id: 3, name: "Amit" }
];
```

```
function Users() {
  return (
    <div>
      {users.map(user => (
        <div key={user.id}>
          {user.name}
        </div>
      ))}
    </div>
  );
}
```

Here:

key={user.id}

uniquely identifies each item.

How does it work?

Suppose initially React has:

1 → Avhi

2 → Rahul

3 → Amit

Then the data changes to:

1 → Avhi

3 → Amit

4 → John

Because React has keys:

1 → same item

3 → same item

2 → removed

4 → new item

React can efficiently determine what changed instead of treating the entire list as completely new.

Why shouldn't we normally use array index?

// Avoid when list can change

```
users.map((user, index) => (  
  <User key={index} />  
));
```

If items are inserted, removed, or reordered, indexes can change and React may associate the wrong component/state with an item.

Prefer:

```
key={user.id}
```

Real-time example

Imagine an e-commerce cart:

Product A

Product B

Product C

If Product B is removed, a stable product ID allows React to correctly understand that Product B was removed, while A and C are still the same items.

Interview Answer

Keys are unique identifiers used when rendering lists in React. They help React identify which items have been added, removed, or changed during reconciliation. I prefer using a stable unique ID from the data, such as `user.id`, instead of the array index, especially when the list can be reordered or modified.

15. What is JSX?

Ans:

JSX stands for JavaScript XML. It is a syntax extension for JavaScript that allows us to write HTML-like UI syntax directly inside JavaScript.

Example:

```
function Welcome() {  
  const name = "Avhi";  
  
  return <h1>Hello {name}</h1>;  
}
```

It looks like HTML:

```
<h1>Hello</h1>
```

but it is actually part of JavaScript code.

How does JSX work?

JSX is transformed into JavaScript that creates React elements.

For example:

```
const element = <h1>Hello Avhi</h1>;  
is transformed conceptually into something like:  
const element = React.createElement(  
  "h1",  
  null,  
  "Hello Avhi"  
);
```

Modern React tooling generally uses the automatic JSX runtime, so you don't necessarily need to import React just to write JSX.

JSX allows JavaScript expressions

You can use `{}`:

```
const name = "Avhi";  
const age = 25;
```

```

return (
  <div>
    <h1>{name}</h1>
    <p>{age}</p>
  </div>
);

```

You can also use expressions:

```
<p>{age} >= 18 ? "Adult" : "Minor"</p>
```

Important: JSX is not HTML

There are some differences.

For example:

```
<div className="container">
```

instead of:

```
<div class="container">
```

And:

```
<button onClick={handleClick}>
```

instead of HTML-style event attributes.

Real-time example

When converting a Figma design into a React application, you might create:

```

function ProductCard({ product }) {
  return (
    <div className="card">
      <img src={product.image} alt={product.name} />
      <h2>{product.name}</h2>
      <p>₹{product.price}</p>
    </div>
  );
}

```

JSX makes this UI structure easier to read and maintain.

Interview Answer

JSX is a syntax extension for JavaScript that allows us to write HTML-like UI structures inside JavaScript. React tooling transforms JSX into JavaScript that creates React elements. It also allows us to embed JavaScript expressions using curly braces, which makes it convenient to build dynamic UIs.

16. What are Props in React?

Ans:

Props, short for properties, are read-only data passed from a parent component to a child component.

They allow components to be reusable and configurable.

Example

```

function App() {
  return <User name="Avhi" role="Frontend Developer" />;
}

```

```

function User({ name, role }) {
  return (
    <div>
      <h2>{name}</h2>
      <p>{role}</p>
    </div>
  );
}

```

```
    </div>
  );
}
```

Here:

Parent

↓

props

↓

Child

The parent passes:

name="Avhi"

role="Frontend Developer"

The child receives them:

```
function User({ name, role })
```

Props can contain different types

String

```
<User name="Avhi" />
```

Number

```
<User age={25} />
```

Boolean

```
<User isAdmin={true} />
```

Object

```
<User user={user} />
```

Array

```
<User skills={skills} />
```

Function

```
<User onDelete={handleDelete} />
```

JSX / Component

```
<Layout header={<Header />} />
```

Function as a prop

This is very important for interviews.

```
function Parent() {
  const handleDelete = (id) => {
    console.log(id);
  };

  return <Child onDelete={handleDelete} />;
}
```

Child:

```
function Child({ onDelete }) {
  return (
    <button onClick={() => onDelete(10)}>
      Delete
    </button>
  );
}
```

This allows the child to communicate an event back to the parent.

The child doesn't directly modify the parent's state. Instead, it calls the function provided by the parent.

Props are read-only

You shouldn't do:

```
function Child({ name }) {  
  // Don't modify props  
  name = "Rahul";  
}
```

Instead, if the child needs its own changing value, use state.

Real-time example

Imagine a reusable button:

```
<Button  
  title="Delete"  
  variant="danger"  
  onClick={handleDelete}  
>
```

The Button component doesn't need to know what "delete" means. It receives configuration through props.

That's what makes it reusable.

Interview Answer

Props are read-only values passed from a parent component to a child component. They are used to make components reusable and configurable. Props can contain strings, numbers, objects, arrays, functions, or even JSX. A child should not modify its props directly; if it needs to change something, it can communicate back to the parent through a callback prop.

17. When Do We Need a Higher-Order Component?

Ans:

A Higher-Order Component, or HOC, is a function that takes a React component as input and returns a new enhanced component.

The basic pattern is:

```
const EnhancedComponent = HOC(OriginalComponent);
```

Think of it as:

Component

↓

HOC

↓

Enhanced Component

Simple Example

```
function withLoading(Component) {  
  return function EnhancedComponent({ loading, ...props }) {  
    if (loading) {  
      return <p>Loading...</p>;  
    }  
  
    return <Component {...props} />;  
  };  
}
```

Now:

```
function UserList({ users }) {  
  return (  
    <div>  
      {users.map(user => (  
        <p key={user.id}>{user.name}</p>  
      ))}  
    )  
  )  
}
```

```
    </div>
  );
}
```

```
const UserListWithLoading = withLoading(UserList);
```

Usage:

```
<UserListWithLoading
  loading={true}
  users={users}
/>
```

The HOC adds loading behavior without changing UserList.

When would you use an HOC?

Historically, HOCs were commonly used to reuse component behavior such as:

- Authentication
- Permissions
- Loading states
- Logging
- Feature flags
- Data subscriptions

For example:

UserComponent



withAuthentication()



ProtectedUserComponent

HOC vs Custom Hook

This is important for modern React.

For **new functional-component code**, custom Hooks are often a cleaner way to reuse logic.

HOC

```
withAuth(UserProfile)
```

Enhances a component.

Custom Hook

```
const { user } = useAuth();
```

Shares logic directly inside the component.

Interview Answer

A Higher-Order Component is a function that takes a component and returns an enhanced component. It was commonly used to reuse cross-cutting component logic such as authentication, permissions, or loading behavior. In modern functional React applications, I would usually prefer custom Hooks for sharing logic because they are generally simpler and avoid adding extra component layers.

18. What Are the Different Phases of the Component Lifecycle?

A React component lifecycle can be understood in three main phases:

Mounting



Updating



Unmounting

1. Mounting

Mounting means the component is being added to the UI for the first time.

Example:

```
function User() {  
  useEffect(() => {  
    console.log("Component mounted");  
  }, []);  
  
  return <h1>User</h1>;  
}
```

The component renders and then the effect runs.

Typical activities:

- Initial rendering
 - Setting up subscriptions
 - Initial data fetching
 - Setting up event listeners
-

2. Updating

Updating happens when something that affects the component changes.

For example:

- State changes
- Props change
- Consumed context changes

```
const [count, setCount] = useState(0);
```

When:

```
setCount(1);
```

React renders again.

An effect with dependencies can respond to the change:

```
useEffect(() => {  
  console.log("count changed");  
}, [count]);
```

3. Unmounting

Unmounting means the component is removed from the UI.

This is where cleanup is important.

```
useEffect(() => {  
  const timer = setInterval(() => {  
    console.log("Running");  
  }, 1000);
```

```
  return () => {  
    clearInterval(timer);  
  };  
}, []);
```

When the component is removed, the cleanup function runs.

Real-time example

Suppose you have a chat component:

Open Chat



Connect WebSocket



User interacts



Component updates



Close Chat



Disconnect WebSocket

The connection should be cleaned up when the component unmounts.

Interview Answer

A React component lifecycle can be understood in three phases: mounting, updating, and unmounting. During mounting, the component is created and rendered. During updating, React re-renders when relevant state, props, or context changes. During unmounting, the component is removed from the UI and we should clean up resources such as subscriptions, timers, and event listeners. With functional components, `useEffect` and its cleanup function are commonly used for this synchronization and cleanup.

19. What are React Suspense and React.lazy? How Do They Enable Code Splitting?

What is Code Splitting?

Code splitting means breaking the application's JavaScript bundle into smaller chunks and loading them when they are needed instead of loading everything initially.

Without code splitting:

Application



Huge JS Bundle



Browser downloads everything

With code splitting:

Application



┌───┴───┴───┐
Home Admin Reports

↓ ↓ ↓

load load load

when when when

needed needed needed

This can improve initial loading performance.

React.lazy

React.lazy allows you to dynamically import a component.

```
const Dashboard = React.lazy(  
  () => import("./Dashboard")  
);
```

The Dashboard code doesn't need to be included in the initial JavaScript execution path in the same way as a normal static import.

Suspense

Suspense lets you specify a fallback UI while something is waiting.

```
<Suspense fallback={<p>Loading...</p>}>  
  <Dashboard />  
</Suspense>
```

Complete example:

```
import { lazy, Suspense } from "react";
```

```
const Dashboard = lazy(  
  () => import("./Dashboard")  
);
```

```
function App() {  
  return (  
    <Suspense fallback={<p>Loading...</p>}>  
      <Dashboard />  
    </Suspense>  
  );  
}
```

How they work together

User needs Dashboard



React.lazy()



Dynamic import()



Dashboard chunk requested



Suspense shows fallback



Chunk loads



Dashboard renders

Real-time example

Imagine an admin application:

Login



Dashboard



Reports



Analytics



Settings

A normal user may never open:

Reports

Analytics

Settings

There is little reason to make all of those feature modules part of the initial loading path.

You can lazy-load them:

```
const Reports = lazy(() => import("./Reports"));
```

```
const Analytics = lazy(() => import("./Analytics"));
```

Then load them when the user navigates to those sections.

Important interview point

Suspense itself doesn't perform code splitting.

React.lazy + dynamic import enables component-level code splitting, while Suspense provides the fallback UI while the lazy component is loading.

Interview Answer

Code splitting means dividing the application's JavaScript into smaller chunks so that we don't have to load everything upfront. React.lazy allows us to dynamically import a component, and Suspense provides a fallback UI while that component is loading. For example, in an admin application I can lazy-load Reports or Analytics pages so their code is fetched when the user actually needs them, reducing the initial JavaScript required by the application.

20. What is React.Fragment and Why Is It Useful?

Ans:

React.Fragment allows us to group multiple elements without adding an extra DOM element.

Normally, a React component must return one parent element.

This won't work:

```
return (  
  <h1>Hello</h1>  
  <p>Welcome</p>  
);
```

You can use a <div>:

```
return (  
  <div>  
    <h1>Hello</h1>  
    <p>Welcome</p>  
  </div>  
);
```

But this adds an unnecessary <div> to the DOM.

Instead, use a Fragment:

```
return (  
  <React.Fragment>  
    <h1>Hello</h1>  
    <p>Welcome</p>  
  </React.Fragment>  
);
```

Or the short syntax:

```
return (  
  <>  
    <h1>Hello</h1>  
    <p>Welcome</p>  
  </>  
);
```

The Fragment doesn't create an additional DOM node.

Why is that useful?

It keeps the DOM cleaner.

Instead of:

```
<div>  
  <h1>Hello</h1>  
  <p>Welcome</p>  
</div>
```

you get:

```
<h1>Hello</h1>
<p>Welcome</p>
```

This can be useful when the extra wrapper would affect:

- CSS
 - Flexbox/Grid layouts
 - DOM structure
 - Accessibility
 - HTML semantics
-

Fragment with keys

One important interview point:

The shorthand:

```
<>
```

cannot receive a key.

If you need a Fragment inside a list, use:

```
<React.Fragment key={item.id}>
  <h2>{item.name}</h2>
  <p>{item.description}</p>
</React.Fragment>
```

Example:

```
items.map(item => (
  <React.Fragment key={item.id}>
    <h2>{item.name}</h2>
    <p>{item.description}</p>
  </React.Fragment>
))
```

Interview Answer

React Fragment allows us to group multiple elements without adding an extra DOM node. I use it when a component needs to return multiple elements but adding a wrapper like a div would be unnecessary or could affect the layout or HTML structure. The shorthand syntax is `<>...</>`, while `React.Fragment` is useful when we need to provide a key, such as when rendering multiple elements inside a list.

21. Explain the types of routers in React

In React, routing is commonly handled using **React Router**. It allows us to display different components based on the URL without doing a full page reload.

Main types/patterns

1. BrowserRouter

Uses the browser's **History API** and normal-looking URLs.

```
import { BrowserRouter } from "react-router-dom";
```

```
<BrowserRouter>
  <App />
</BrowserRouter>
```

Example URLs:

```
/          → Home
/about     → About
/products  → Products
```

This is the most commonly used router for standard web applications.

2. HashRouter

Uses the URL hash (#) for routing.

```
import { HashRouter } from "react-router-dom";
```

```
<HashRouter>
  <App />
</HashRouter>
```

URLs look like:

```
/#/
```

```
/#/about
```

```
/#/products
```

It's useful when server-side configuration for SPA routes isn't available, because everything after # isn't sent to the server as part of the request path.

3. MemoryRouter

Keeps routing information **in memory instead of the browser URL**.

```
import { MemoryRouter } from "react-router-dom";
```

```
<MemoryRouter>
  <App />
</MemoryRouter>
```

It's mainly useful for:

- Testing
 - Non-browser environments
 - Specialized embedded applications
-

What about createBrowserRouter?

Modern React Router also provides **data routers**, such as:

```
createBrowserRouter()
```

```
createHashRouter()
```

```
createMemoryRouter()
```

For example:

```
const router = createBrowserRouter([
  {
    path: "/",
    element: <Home />
  },
  {
    path: "/about",
    element: <About />
  }
]);
```

Interview Answer

React itself doesn't provide routing; we commonly use React Router. The main router types are BrowserRouter, HashRouter, and MemoryRouter. BrowserRouter is commonly used for web applications and uses the browser History API, HashRouter uses the URL hash, and MemoryRouter keeps the routing state in memory and is commonly useful for testing or non-browser environments. Modern React Router also provides data-router APIs such as createBrowserRouter.

22. How do you use the ternary operator for conditional rendering in React?

The **ternary operator** is used when we want to render one thing if a condition is true and another thing if it's false.

Syntax:

condition ? trueValue : falseValue

Example

```
function User({ isLoggedIn }) {  
  return (  
    <div>  
      {isLoggedIn ? (  
        <h2>Welcome back!</h2>  
      ) : (  
        <h2>Please login</h2>  
      )}  
    </div>  
  );  
}
```

If:

isLoggedIn = true

React renders:

Welcome back!

If:

isLoggedIn = false

React renders:

Please login

Real-time example

Suppose you have a loading state:

```
return (  
  <div>  
    {loading ? (  
      <Spinner />  
    ) : (  
      <UserList users={users} />  
    )}  
  </div>  
);
```

So:

loading = true

↓

Spinner

loading = false

↓

UserList

Interview Answer

I use the ternary operator in JSX when I need to conditionally render one of two possible UI elements. For example, I can show a loading spinner when loading is true and the actual data when it's false.

23. What is the purpose of the && operator in JSX?

The `&&` operator is commonly used when we want to **render something only when a condition is true**.

Example

```
function App({ isLoggedIn }) {  
  return (  
    <div>  
      {isLoggedIn && <button>Logout</button>}  
    </div>  
  );  
}
```

If:

`isLoggedIn = true`

the button is rendered.

If:

`isLoggedIn = false`

nothing is rendered for that expression.

How does it work?

JavaScript evaluates:

`true && "Hello"`

as:

`"Hello"`

But:

`false && "Hello"`

evaluates to:

`false`

React doesn't render the boolean false, so nothing appears.

Real-time example

Show an error only when there is an error:

```
{error && <p>{error}</p>}
```

Show an admin button only for admins:

```
{user.isAdmin && <button>Delete User</button>}
```

Important interview point

Be careful with numbers:

```
{count && <p>There are items</p>}
```

If:

`count = 0`

the expression evaluates to 0, and React can render that 0 on the screen.

If you specifically want to check for a positive count:

```
{count > 0 && <p>There are items</p>}
```

&& vs ternary

Use `&&` when you have **only one condition**:

```
{isAdmin && <AdminPanel />}
```

Use a ternary when you need **either of two outputs**:

```
{isAdmin ? <AdminPanel /> : <UserPanel />}
```

Interview Answer

The `&&` operator is used for conditional rendering when I only want to render something if a condition is true. For example, `{isAdmin && <AdminPanel />}` renders the `AdminPanel` only when `isAdmin` is true. If I need to render one component when true and another when false, I use the ternary operator instead.

